# Data Structures and Algorithms

## Lab 08 - Recursion

**Instructor: Sidrat-ul-Muntha**

---

**READ IT FIRST**

Prior to start solving the problems in this assignments, please give full concentration on following points.

1. WORKING – This is individual lab. If you are stuck in a problem contact your teacher, but, in mean time start doing next question (don't waste time).

2. DEADLINE – One Week after assigned this.

3. SUBMISSION – This assignment needs to be submitted in a soft copy.

4. WHERE TO SUBMIT – Please visit your LMS.

5. **HOW TO SUBMIT** – Rename all .java files according to the task number (e.g., Task1.java, Task2.java). Place all required .java files in a .zip file named 000-00-0000_assignment.zip and submit the ZIP file on the LMS.

---

## Learning Objectives

By the end of this lab, students will be able to:

- Implement recursive solutions with a correct base case and recursive case for problems such as power, sum of digits, GCD, Fibonacci, and finding the maximum in an array.
- Trace recursive calls and explain how a recursive method reaches its base case and returns its result.
- Apply recursion to data structures by finding the length and reversing a Singly Linked List, sorting a Stack, and reversing a Queue.

### Task 1: Power of a Number

Write a recursive method that computes x raised to the power n (x^n), where x and n are entered by the user.

Implement the following methods

- power(x, n) – Return x^n using recursion.

Demonstration
In the main() method, take x and n as input and display the result.

---

## Task 2: Sum of Digits

Write a recursive method that finds the sum of the digits of a positive integer.

Implement the following methods

- sumOfDigits(n) – Return the sum of the digits of n.

Demonstration
In the main() method, input a number and display its digit sum (e.g., 1234 → 10).

---

## Task 3: GCD Using Recursion

Find the Greatest Common Divisor of two numbers using the Euclidean algorithm.

Implement the following methods

- gcd(a, b) – Return the GCD of a and b recursively.

Demonstration
In the main() method, input two numbers and display their GCD.

---

## Task 4: Fibonacci Series

Find the nth Fibonacci number using recursion.

Implement the following methods

- fib(n) – Return the nth Fibonacci number recursively.

Demonstration
In the main() method, input n and display the Fibonacci series from the 0th term up to the nth term by calling fib() for each term.

---

## Task 5: Maximum Element in an Array

Find the largest element of an integer array using recursion.

Implement the following methods

- findMax(arr, n) – Return the maximum among the first n elements of the array recursively.

Demonstration
In the main() method, create an array of integers and display its maximum element.

---

## Task 6: Length of a Linked List

Find the number of nodes in a linked list using recursion.

Implement the following methods

- length(node) – Return the number of nodes recursively.

Demonstration
In the main() method, create a list with several nodes and display its length.

---

## Task 7: Reverse a Singly Linked List

Reverse a singly linked list using recursion.

Implement the following methods

- reverse(node) – Recursively reverse the list and return the new head.

Demonstration
In the main() method, create a list with several nodes and print it before and after reversal.

---

## Task 8: Sort a Stack Using Recursion

Sort a stack using recursion only (no loop constructs).

Implement the following methods

- sortStack(stack) – Recursively sort the stack.
- sortedInsert(stack, element) – Insert an element into the stack at its correct position.

Demonstration
In the main() method, push several unsorted elements, then display the stack before and after sorting.

---

## Task 9: Reverse a Queue Using Recursion

Reverse the elements of a queue using recursion only (no loop constructs and no extra stack).

Implement the following methods

- reverseQueue(queue) – Recursively reverse the queue.

Demonstration
In the main() method, enqueue several elements, then display the queue before and after reversal.

---

## Task 10: Palindrome String

Check whether a string is a palindrome using recursion, ignoring uppercase and lowercase differences. For example, "racecar" is a palindrome, while "cake" is not.

Implement the following methods

- isPalindrome(s, left, right) – Return true if the string is a palindrome, otherwise false, using recursion.

Demonstration
In the main() method, input a string and display whether it is a palindrome